BÖLÜM 6: GÜVENILIRLIK VE HATA TOLERANSI — ÇÖKÜŞ İÇIN TASARLAMAK

⏱ 9 dk • DERS 22

Parça 22: Devre Kesiciler (Circuit Breakers), Bölmeler (Bulkheads) ve Üstel Geri Çekilme (Exponential Backoff)

Kendi kendinize DDoS saldırısı yapmayı engellemek ve bir savaş gemisi gibi hasarı izole etmek.

🔥 **SAHADAN YAŞANMIŞ KRİZ HİKAYESİ**

Bir ödeme kuruluşunda döviz çevrim servisinde 100ms'lik bir gecikme yaşandı. Ana ödeme servisi agresif bir yeniden deneme (retry) kuralına sahipti: Yanıt gelmezse hemen 5 kez tekrar dene! Gecikme arttıkça gelen 1.000 kullanıcı isteği anında 6.000 eşzamanlı isteğe dönüştü ve zaten zorlanan döviz servisini bombaladı. Döviz servisi CPU %100 ile çöktü. Artık anında 503 hatası dönüyordu ama ana servis 5 kez yeniden denemeye tam gaz devam etti! Döviz servisi yeniden başlatıldığında bile biriken milyonlarca deneme isteği ayağa kalkmasına fırsat vermeden onu tekrar yıktı. Mühendisler kendi elleriyle kendilerine bir DDoS silahı inşa etmişlerdi.

Zihniyet Dönüşümü: Kod Yazarlığından Mimarlığa

Zorlanan bir bağımlılığı hemen ve durmaksızın yeniden denemek, hasta bir atı daha hızlı koşsun diye kırbaçlamaya benzer; onu öldürürsünüz. Dayanıklı dağıtık sistemler; denemeleri zamana yaymak için **Jitter ile Üstel Geri Çekilme (Exponential Backoff)**, can çekişen servise giden musluğu kapatıp nefes almasını sağlamak için **Devre Kesici (Circuit Breaker)** ve yangının diğer odalara sıçramaması için **Bölmeler (Bulkheads)** kullanır.

Temel Mimari İlkeler ve Zihinsel Modeller

Devre Kesici (Circuit Breaker) Durum Makinesi

- **Kapalı (Closed - Normal):** İstekler geçer. Hatalar sayılır.
- **Açık (Open - Devre Attı):** Hata oranı eşiği (örn: son 10 saniyede %50 başarısızlık) aşıldığında devre AÇILIR. İstekler aşağıya hiç gönderilmeden anında başarısız sayılır veya fallback döner. Arka uç nefes alır.
- **Yarı Açık (Half-Open - Test):** 30 saniye sonra küçük bir test trafiği gönderilir. Başarılıysa Kapalı'ya döner; başarısızsa tekrar Açık duruma geçer.

Bölme (Bulkhead) Deseni (Gemi Gövdesi İzolasyonu)

Bir savaş gemisi torpido yediğinde su geçirmez bölmeleri kapatarak tek bir odanın batmasını sağlar, tüm gemi batmaz. Yazılımda da iş parçacığı (thread) ve bağlantı havuzlarını ayırın: Arka plan raporlama thread'leri asla müşteri ödeme thread'leriyle aynı havuzu paylaşmamalıdır.

Jitter ile Üstel Geri Çekilme

Yeniden deneme formülü: $text{bekleme} = 2^{text{deneme}} + text{rastgele_sapma}$. Jitter istemcilerin aynı milisaniyede hep birlikte hücum etmesini engeller.

Kaçınılması Gereken Anti-Pattern'ler ve Tuzaklar

- **Kör Yeniden Deneme:** `while (error) { retry(); }` — Sistemi kurtulamaz bir ölüm döngüsüne sokmanın garantili yolu.
- **Zaman Aşımı Olmayan İstekler:** Bir socket bağlantısını açık bırakıp thread'i sonsuza dek askıda tutmak.
- **Ortak Thread Havuzu Tuzağı:** Hem kritik API isteklerini hem de 10 dakikalık PDF üretimlerini aynı thread havuzundan beslemek.

> 🎯 **MİMARIN EYLEM PLANI: BU HAFTAKİ SOMUT ADIMLAR**
>
> - **1. Adım:** Dış dünya ile konuşan tüm istemcilerde Circuit Breaker (Resilience4j, Polly veya Envoy) kur.
> - **2. Adım:** Tüm yeniden deneme kodlarına rastgele sapmalı (jitter) üstel geri çekilme ekle.
> - **3. Adım:** Ödeme ve sepet bağlantı havuzlarını diğer tali servislerden bölmelerle (bulkhead) ayır.

Mimari Çıkarımlar ve Hile Tablosu

- Karşı taraf boğulurken hızla başarısız olun (fail fast); kullanıcıyı ekranda 60 saniye bekletmeyin.
- Jitter içermeyen bir yeniden deneme politikası, palto giymiş bir DDoS saldırısından farksızdır.
- Bölmeler ana gelir kanallarınızı tali özelliklerin kurbanı olmaktan korur.

> 🎙️ **UDEMY & NOTEBOOKLM TARTIŞMA VE ÖDEV SORUSU**
>
> **Ders Değerlendirmesi ve Ödev:** Kodunuzdaki HTTP istemcilerini açın: Her istekte açıkça tanımlanmış bir zaman aşımı var mı? Hata durumunda yeniden deneme stratejiniz nasıl çalışıyor?